

AI 201 Condensed Reviewer

Short edition of the full reviewer: same lectures, plainer wording, repeated material removed. Bold terms are identification targets. “Traps” are the usual True/False items. Proof numbers match the full reviewer and the answer keys. Items marked (*beyond slides*) are standard results that are not on the slides.

Lecture 1: What is AI

AI (AIMA): the study and construction of **rational agents**.

Four approaches

	Like humans	Rationally
Think	Cognitive Modeling	Laws of Thought
Act	Turing Test	Rational Agent

- **Turing Test (1950):** an interrogator types to a hidden respondent for 30 minutes. If the machine cannot be told apart from a human, it is considered intelligent. It tests behavior, not thinking.
- **Six capabilities** needed to pass (also the six subfields of AI): natural language processing, knowledge representation, automated reasoning, machine learning, computer vision/speech, robotics.
- **Cognitive modeling:** build testable theories of the human mind (introspection, experiments, fMRI/EEG).
- **Laws of thought:** Aristotelian logic. A **sylogism** has a major premise, a minor premise, and a conclusion; correct premises give a correct conclusion.
- **Rational agent:** acts to achieve its goals given its beliefs. Correct inference is only part of it, because sometimes there is no provably correct action but the agent must still act.

Rationality terms

- **Perfect rationality:** always doing the right thing. Not feasible; computation is too costly.
- **Bounded rationality (Simon, 1957):** decisions are limited by cognitive ability, time, and imperfect information.
- **Satisficing:** accepting a good-enough result instead of the optimal one.
- **Any-time algorithm:** can be interrupted at any time; result quality increases with time.
- **Any-space algorithm:** works with arbitrarily low memory and is still optimal at termination.

History

- **Inception (1943–1956):** 1943 McCulloch and Pitts, model of the neuron. 1950 Minsky and Edmonds (slide: “Edwards”), SNARC, first neural network computer. 1950 Turing Test. 1952 Strachey and Samuel, checkers programs. 1956 Dartmouth Workshop; McCarthy coins “Artificial Intelligence”. 1956 Simon’s Logic Theorist.
- **Early enthusiasm (1952–1969):** 1957 Rosenblatt, Perceptron Convergence Theorem. 1958 McCarthy, Lisp. 1959 Newell and Simon, General Problem Solver. 1960 Widrow and Hoff, Adaline. 1963 microworlds (Blocks World). 1965 Robinson, resolution.
- **Dose of reality (1966–1973):** Minsky and Papert show the perceptron cannot learn XOR. Lighthill Report (UK, 1973) and ALPAC (US) lead to the AI Winter.

Cause: combinatorial explosion.

- **Expert systems (1969–1986):** 1969 DENDRAL (Buchanan), molecular structure. 1975 MYCIN (Shortliffe), 600 rules, backward chaining, bacterial infections. 1982 R1 (McDermott), first commercial expert system. 1982 Japan’s Fifth Generation Project.
- **Neural networks and probability (1986–now):** 1982 VC dimension (Vapnik, Chervonenkis). 1986 backpropagation (Rumelhart, Hinton, Williams). 1988 Bayesian networks (Pearl); reinforcement learning (Sutton). 1992 SVM (Vapnik). 2011 IBM Watson. 2012 AlexNet (Krizhevsky), start of deep learning. 2014 GAN (Goodfellow). 2015 ResNet152 beats humans on ImageNet. 2016 AlphaGo. 2017 Transformer.

Problem domains

- From puzzles to vision, knowledge content goes from poor to rich, data rate from low to very high, and response time from hours to real time.
- **Chess:** more than 10^{123} possible moves. Deep Thought (CMU), first Grandmaster-level program. Deep Blue lost to Kasparov in 1996, won in 1997.
- **Go:** more than 10^{361} moves. 2007 MCTS. 2015 AlphaGo beat Fan Hui. 2017 AlphaZero beat Ke Jie. **AlphaZero:** one reinforcement learning algorithm for chess, Go, and Shogi, given only the rules.
- **Speech:** Harpy, Hearsay, HWIM, Sphinx; produced HMM learning, beam search, CTC.
- **Vision:** about 2 orders of magnitude more complex than speech.

Five Laws of Intelligent Action

1. Bounded rationality implies opportunistic search (aim for good enough).
2. A physical symbol system is necessary and sufficient for intelligent action.
3. The magic number is $70,000 \pm 20,000$ chunks (an expert’s knowledge; about a decade of study).
4. Search compensates for lack of knowledge (Deep Blue: 200 million moves per second).
5. Knowledge compensates for lack of search (Sphinx: removing syntax knowledge raised errors from 4% to 30%).

Risks of AI: lethal autonomous weapons, surveillance and persuasion, biased decisions, employment, safety-critical applications, cybersecurity.

Traps

- The Turing Test is *acting* humanly, not thinking humanly.
- Use the slide’s years (SNARC 1950, Samuel’s checkers 1952).
- Law 4 example: Deep Blue. Law 5 example: Sphinx.

Lecture 2: Intelligent Agents

Agent: perceives its environment through sensors and acts on it through actuators (effectors). **Percept:** the sensor

input at one instant. **Percept sequence:** the full history of percepts so far.

Rationality

- **Performance measure:** how success is scored. Define it by the result wanted in the environment, not by how the agent should behave.
- What is rational depends on four things: performance measure, percept sequence, knowledge of the environment, available actions.
- **Ideal rational agent:** for each percept sequence, picks the action expected to maximize the performance measure, using the percept sequence and built-in knowledge. Its behavior is a mapping from percept sequences to actions.
- Rational is not perfect: rationality maximizes *expected* performance, perfection maximizes *actual* performance.
- **Information gathering / exploration:** acting to get useful future percepts is part of rationality.
- **Autonomy:** behavior is determined by the agent's own experience, not only built-in knowledge (a clock has none).
- **Agent = architecture + program.** The **agent program** implements the percept-to-action mapping; the **architecture** is the computing device.
- **PEAS:** Performance, Environment, Actuators, Sensors. Example, part-picking robot: % of parts in correct bins; conveyor belt and bins; jointed arm and hand; camera and joint sensors.

Agent types

Type	How it decides; limitation
Table-driven	Looks up the whole percept sequence in a table. Table is too large (10^{123} entries for chess); no autonomy.
Simple reflex	Condition-action rules on the current percept only. Ignores history.
Model-based reflex	Keeps internal state using a transition model (how the world changes) and a sensor model (how state shows up in percepts). Handles partial observability.
Goal-based	Model + goals; considers the future. Flexible, less efficient.
Utility-based	Model + utility function (state $\rightarrow$ real number). Resolves conflicting goals.
Learning agent	Any of the above plus learning.

Learning agent parts: performance element (selects actions), learning element (improves it), critic (scores against a performance standard), problem generator (suggests exploratory actions).

Environment properties

- **Fully / partially observable:** sensors give the complete state, or not.
- **Single / multi-agent.**
- **Deterministic / stochastic:** next state is fully determined by current state and action, or not.
- **Episodic / sequential:** action quality depends only on the current episode, or on earlier actions too.
- **Static / dynamic:** environment can change while the agent deliberates. **Semi-dynamic:** environment does not change but the score does (chess with a clock).
- **Discrete / continuous:** limited number of distinct

percepts and actions, or not.

Task	Obs.	Agents	Det.	Epis.	Static	Disc.
Crossword	Fully	Single	Det.	Seq.	Static	Disc.
Chess, clock	Fully	Multi	Det.	Seq.	Semi	Disc.
Poker	Part.	Multi	Stoch.	Seq.	Static	Disc.
Backgammon	Fully	Multi	Stoch.	Seq.	Static	Disc.
Taxi driving	Part.	Multi	Stoch.	Seq.	Dyn.	Cont.
Med. diagnosis	Part.	Single	Stoch.	Seq.	Dyn.	Cont.
Image analysis	Fully	Single	Det.	Epis.	Semi	Cont.
Picking robot	Part.	Single	Stoch.	Epis.	Dyn.	Cont.
Refinery ctrl.	Part.	Single	Stoch.	Seq.	Dyn.	Cont.
English tutor	Part.	Multi	Stoch.	Seq.	Dyn.	Disc.

Traps

- Simple reflex uses the current percept; table-driven uses the whole percept sequence.
- A rational agent can still get a bad outcome.
- Backgammon is fully observable but stochastic. Poker is partially observable.
- Hardest case: partially observable, multi-agent, stochastic, sequential, dynamic, continuous (taxi).

Lecture 3A: Blind Search

A **problem-solving agent** is goal-based: it **formulates** the problem, **searches** for an action sequence, then **executes** it.

Problem types

- **Single-state:** knows its exact state and the effects of actions.
- **Multiple-state:** knows only a set of possible states; still knows action effects.
- **Contingency:** the right action depends on what happens during execution (driving, games). Search and execution are interleaved.
- **Exploratory:** does not know action effects; must experiment (reinforcement learning).

Search problem

Defined by: initial state, **operators** (actions), **goal test**, **path cost**.

- **State space:** all states reachable from the initial state. **Successor function** $S(x)$: states reachable from x in one action.
- **Solution:** a path from the initial state to a goal state.
- **Expanding:** generating a node's successors. **Frontier (fringe):** nodes waiting to be expanded, kept in a queue.
- **Search strategy:** the rule for which node to expand next. In GENERAL-SEARCH only the queuing function differs between strategies.
- **Node:** state, parent, operator, depth, path cost.
- **Blind (uninformed):** no information on distance to the goal. **Heuristic (informed):** uses problem-specific knowledge.
- Toy problems: 8-puzzle (step cost 1), 8-queens (path cost 0; incremental vs. complete-state formulation), route finding, Blocks World (Sussman's Anomaly = subgoal interaction).

Four criteria: completeness (finds a solution if one exists), time, space, optimality/admissibility (finds the lowest-cost

solution).

The six strategies

b branching factor, d depth of shallowest solution, m maximum depth, ℓ depth limit, C^* optimal cost, ϵ minimum step cost.

	Complete	Optimal	Time	Space
BFS	Yes	If equal costs	b^d	b^d
UCS	Yes, costs $\geq \epsilon$	Yes	$b^{1+\lceil C^*/\epsilon \rceil}$	same
DFS	No	No	b^m	bm
DLS	If $\ell \geq d$	No	b^ℓ	$b\ell$
IDS	Yes	If equal costs	b^d	bd
Bidirectional	Yes [†]	Yes [†]	$b^{d/2}$	$b^{d/2}$

[†] if both directions use BFS or UCS.

- **BFS:** new nodes go to the end of the queue (FIFO); shallowest first.
- **UCS (Dijkstra, 1959):** expand the lowest path cost $g(n)$. BFS is UCS with $g(n) = \text{DEPTH}(n)$.
- **DFS:** new nodes go to the front (LIFO); deepest first. Can run forever on infinite depth.
- **DLS:** DFS with a depth cut-off ℓ .
- **IDS (Slate and Atkin, 1977):** DLS with $\ell = 0, 1, 2, \dots$. Preferred when the space is large and the solution depth is unknown. Repeated work is small.
- **Bidirectional (Pohl, 1969):** search from start and goal at once; needs predecessors.

UCS example

S-A 1, S-B 5, S-C 15, A-G 10, B-G 5, C-G 5. Expand S: A(1), B(5), C(15). Expand A: B(5), G(11), C(15). Expand B: G(10), G(11), C(15). Remove G(10): solution S-B-G, cost 10. The goal test is applied when a node is removed for expansion, not when generated.

Traps

- BFS is optimal only when path cost is non-decreasing with depth (for example equal step costs).
- IDS: BFS's time, DFS-like space.
- DFS is neither complete nor optimal.
- Slide: BFS's bigger problem is time (AIMA says memory). Slide 34 prints DFS storage as b^m ; the table's bm is the standard result.

Lecture 3B: Heuristic Search

Knowledge enters search only through the queuing function, as an **evaluation function** that ranks nodes.

- **Best-first search:** expand the node with the best evaluation value.
- **Heuristic $h(n)$:** estimated cost of the cheapest path from n to a goal; $h(\text{goal}) = 0$. h_{SLD} = straight-line distance.
- **Greedy search:** uses $h(n)$ only. Not complete, not optimal.
- **A*:** uses $f(n) = g(n) + h(n)$, where $g(n)$ is the cost so far. $f(n)$ estimates the cheapest solution through n .
- **Admissible:** never overestimates, $h(n) \leq h^*(n)$, with h^* the true cost to the goal.
- **Consistent / monotonic:** $h(n) \leq c(n, n') + h(n')$ for every successor n' (triangle inequality). Then f never decreases along a path. Consistent implies admissible,

not the reverse.

- **Path max:** $f(n') = \max(f(n), g(n') + h(n'))$ makes an admissible heuristic monotonic.
- **Dominance:** h_2 dominates h_1 if $h_2(n) \geq h_1(n)$ for all n ; A* then expands fewer nodes. The max of admissible heuristics is admissible.
- **Effective branching factor b^* :** solves $N = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$. Good heuristics have b^* near 1.
- **Optimally efficient:** no other optimal algorithm is guaranteed to expand fewer nodes than A* (Dechter and Pearl, 1983).
- **IDA* (Korf):** depth-first searches with an f -limit that rises each iteration to the smallest f that exceeded the last limit. Linear space.
- **OPEN / CLOSED:** nodes waiting for expansion / nodes already expanded.

A* properties: complete (locally finite graph, step costs $\geq \delta > 0$); optimal if h is admissible; time and space exponential; memory is the main drawback. A* cannot solve the 15-puzzle on most machines; IDA* can, but not the 24-puzzle.

8-puzzle heuristics

h_1 = number of misplaced tiles. h_2 = Manhattan distance (sum of each tile's horizontal + vertical distance to its goal square). Both admissible; h_2 dominates h_1 . Slide example: $h_1 = 7$, $h_2 = 18$.

Arad to Bucharest

h_{SLD} : Arad 366, Sibiu 253, Timisoara 329, Zerind 374, Fagaras 178, Oradea 380, Rimnicu Vilcea 193, Pitesti 98, Craiova 160.

Greedy: Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest, cost $140 + 99 + 211 = 450$. Not optimal.

A* (expand lowest $f = g + h$):

Expand	Children, $f = g + h$
Arad 366	Sibiu $140 + 253 = 393$; Timisoara 447; Zerind 449
Sibiu 393	Rimnicu $220 + 193 = 413$; Fagaras $239 + 178 = 417$; Arad 646; Oradea 671
Rimnicu 413	Pitesti $317 + 98 = 415$; Craiova 526; Sibiu 553
Pitesti 415	Bucharest $418 + 0 = 418$; Rimnicu 607; Craiova 615
Fagaras 417	Bucharest 450; Sibiu 591
Bucharest 418	Goal selected; stop

Path: Arad, Sibiu, Rimnicu Vilcea, Pitesti, Bucharest, cost **418**. A* stops when a goal is *selected for expansion*, not when it is generated. (The slide misprints Oradea as $146 + 380 = 526$; correct is $291 + 380 = 671$.)

Traps

- $h = 0$ is admissible; A* then becomes UCS.
- A* ties are broken in favor of a goal node.
- A* and IDA* are both exponential in time; they differ in space.

Lecture 3C: Adversarial Search

A game is a search problem with an opponent who acts to reduce the agent's pay-off. The agent must plan for every reply (a contingency problem).

- **Game:** initial state S_0 , To-Move(s), ACTIONS(s), ISTerminal(s), UTILITY(s, p) (pay-off function).

- **Perfect information** = fully observable. **Position** = state. **Move** = action.
- **MAX / MIN**: MAX maximizes the pay-off and moves first; MIN minimizes it.
- **Ply**: the slide says one move or one ply = two half-moves (AIMA: ply = one half-move).
- **Evaluation function**: estimate of the pay-off of a non-terminal state. **Cut-off test**: replaces the terminal test when depth is limited.
- **Base-level / meta-level**: effort solving the problem vs. effort deciding what to do. Meta-level time must be repaid by base-level savings.

Minimax

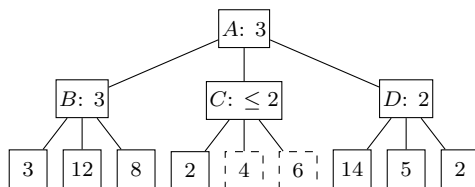
$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{terminal} \\ \max_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{MAX to move} \\ \min_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{MIN to move} \end{cases}$$

Generate the tree to the terminal states, score them, then back values up: MIN nodes take the lowest child, MAX nodes the highest. Assumes a perfect opponent. Depth-first: time $O(b^m)$, space linear. With limited depth (H-Minimax), use an evaluation function and a cut-off test.

Alpha-beta pruning

Returns the same value as minimax but skips branches that cannot change the result.

- α : best (highest) value found so far for MAX ("at least"). β : best (lowest) value found so far for MIN ("at most").
- At a MAX node: prune when $v \geq \beta$. At a MIN node: prune when $v \leq \alpha$.
- Depends on move ordering. Perfect ordering $O(b^{m/2})$, random $O(b^{3m/4})$, no pruning $O(b^m)$.



$B = \min(3, 12, 8) = 3$, so $\alpha = 3$ at the root. C 's first leaf is 2, so $C \leq 2 \leq \alpha$: prune 4 and 6. D : 14 and 5 are both > 3 , no prune; then 2, so $D = 2$. Root = $\max(3, \leq 2, 2) = 3$. Two of nine leaves pruned.

Search refinements

- **Horizon effect**: an unavoidable threat is pushed past the search depth by delaying moves, so the program does not see it.
- **Secondary search**: search deeper under the chosen move to check for this.
- **Quiescent position**: evaluation changes slowly from ply to ply. **Quiescence search** extends non-quiescent positions.
- **Killer heuristic**: try early the moves that were good for the opponent.
- **Singular extension (Anantharaman et al., 1990)**: search deeper when one move is far better than the rest (Deep Thought).
- **ProbCut (forward pruning)**: prunes nodes that are *probably* outside the (α, β) window; alpha-beta prunes only those *provably* outside.

- **Chess**: branching factor about 35. Material: pawn 1, knight/bishop 3, rook 5, queen 9. Linear evaluation $\text{EVAL}(s) = \sum_i w_i f_i(s)$ assumes independent features.

Monte Carlo Tree Search

For games like Go (branching factor above 360, no good evaluation function). Value of a state = average result of many simulated games (**playouts**).

1. **Selection**: follow the selection policy from the root to a leaf.
2. **Expansion**: add a new child.
3. **Simulation**: play out to the end from that child (not stored in the tree).
4. **Back-propagation**: update the nodes up to the root.

Returns the move with the *most playouts*. The selection policy balances exploration and exploitation.

Expectiminimax

Minimax plus **chance nodes**, whose value is the probability-weighted average of the children: $\sum_r P(r) \cdot \text{value}(\text{RESULT}(s, r))$.

Slide example ($P = 0.9, 0.1$): leaves (2, 3) vs. (1, 4) give 2.1 vs. 1.3, choose a_1 . Leaves (20, 30) vs. (1, 400) give 21 vs. 40.9, choose a_2 . Same ordering of leaves, different decision. So the evaluation function must be a **positive linear transformation** of the probability of winning.

Traps

- Alpha-beta changes efficiency only, never the value.
- MCTS returns the most-visited move, not the best win ratio.
- In minimax only the order of values matters; in expectiminimax the magnitudes matter.

Lecture 4: Constraint Satisfaction

A **CSP** has variables X , domains D , and constraints C . A solution is an assignment of values, not a sequence of actions.

- **Constraint**: a pair $\langle \text{scope}, \text{rel} \rangle$: the variables involved and the allowed value combinations.
- **Consistent**: violates no constraint. **Complete**: every variable assigned. **Solution**: consistent and complete. **Partial solution**: consistent partial assignment.
- **Unary / binary / higher-order / global constraints**: 1, 2, 3+, or any number of variables (*Alldiff*).
- **Precedence**: $T_1 + d_1 \leq T_2$. **Disjunctive**: $(A + 10 \leq B)$ or $(B + 10 \leq A)$.
- **Constraint graph**: nodes are variables, links are binary constraints.
- **Backtracking search**: depth-first search that assigns one variable at a time. Assignments are commutative, so only one variable is considered per node.
- **Forward checking**: after each assignment, remove inconsistent values from unassigned neighbors; stop if a domain becomes empty.
- **Arc consistency**: $X \rightarrow Y$ is consistent if every value of X has some allowed value in Y . Values are deleted from the tail X .
- **AC-3**: queue of all arcs. Pop (X_i, X_j) ; REVISE deletes values of D_i with no support in D_j . If D_i changed, re-add

arcs (X_k, X_i) for the other neighbors. Empty domain means failure.

- **Cutset conditioning:** assign a set of variables (in all ways) so the rest of the graph is a tree.

Complexities

- General CSP: $O(d^n)$ assignments; NP-complete.
- Independent subproblems of size $c: \frac{n}{c} d^c$, linear in n . Example $n = 80, d = 2, c = 20: 2^{80}$ (4 billion years) vs. 4×2^{20} (0.4 s).
- Tree-structured CSP: $O(nd^2)$. Cutset of size $c: O(d^c(n - c)d^2)$.
- (*Beyond slides*) AC-3 is $O(cd^3)$; MRV, degree, and least-constraining-value orderings.

Tree algorithm: order variables root to leaves; for $j = n$ down to 2 make $\text{PARENT}(X_j) \rightarrow X_j$ consistent; then assign from $j = 1$ to n . No backtracking.

Australia map coloring

Variables WA, NT, Q, NSW, V, SA, T; colors R, G, B; neighbors differ. SA touches WA, NT, Q, NSW, V. Removing SA leaves a tree, so the cutset is {SA}.

Forward checking:

	WA	NT	Q	NSW	V	SA
WA = R	R	GB	RGB	RGB	RGB	GB
Q = G	R	B	G	RB	RGB	B
V = B	R	B	G	R	B	$\emptyset$

SA is empty, so backtrack. Forward checking missed that NT and SA were both {B} after $Q = G$, because it does not compare two unassigned variables.

Arc consistency after $WA = R, Q = G$: $NSW \rightarrow SA$ removes B from NSW; then $V \rightarrow NSW$ removes R from V; $SA \rightarrow NT$ removes B from SA, leaving it empty. Failure is found one step earlier.

Traps

- In $X \rightarrow Y$, delete from X .
- Consistent is not the same as a solution; it must also be complete.
- Path cost is irrelevant; every solution is at depth n .

Lecture 6A: Linear Classifiers

A linear classifier separates classes with a hyperplane. It makes no assumption about the data distributions. Advantages: simplicity and low computational cost.

Gradient descent

$$\theta_{\text{new}} = \theta_{\text{old}} - \mu \frac{\partial J(\theta)}{\partial \theta}, \quad \mu > 0$$

Move against the gradient to reduce the cost J . It stops where the gradient is zero: a local minimum, the global minimum, or a saddle point. For the quadratic cost $J = b - \mathbf{p}^T \theta + \frac{1}{2} \theta^T R \theta$ it converges if $\mu < 2/\lambda_{\max}$ ($\lambda_{\max}$ = largest eigenvalue of R). For other costs the eigenvalues of the **Hessian** decide. Elliptical cost contours give a zigzag path.

Decision hyperplane

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = 0$$

$\mathbf{w}$ = weight vector (orthogonal to the hyperplane), w_0 = threshold. Distance of $\mathbf{x}$ from the hyperplane: $|g(\mathbf{x})|/\|\mathbf{w}\|$; of the origin: $|w_0|/\|\mathbf{w}\|$. **Linearly separable:** some $\mathbf{w}^*$ gives $\mathbf{w}^{*T} \mathbf{x} > 0$ for all of ω_1 and < 0 for all of ω_2 .

Perceptron

Cost over the set Y of misclassified vectors:

$$J(\mathbf{w}) = \sum_{\mathbf{x} \in Y} \delta_{\mathbf{x}} \mathbf{w}^T \mathbf{x}, \quad \delta_{\mathbf{x}} = \begin{cases} -1 & \mathbf{x} \in \omega_1 \\ +1 & \mathbf{x} \in \omega_2 \end{cases}$$

$J \geq 0$, and $J = 0$ when nothing is misclassified. Batch update: $\mathbf{w}(t+1) = \mathbf{w}(t) - \rho_t \sum_{\mathbf{x} \in Y} \delta_{\mathbf{x}} \mathbf{x}$.

One sample at a time (exam calculation):

$$\mathbf{w}(t+1) = \begin{cases} \mathbf{w}(t) + \rho \mathbf{x} & \mathbf{x} \in \omega_1, \mathbf{w}^T \mathbf{x} \leq 0 \\ \mathbf{w}(t) - \rho \mathbf{x} & \mathbf{x} \in \omega_2, \mathbf{w}^T \mathbf{x} \geq 0 \\ \mathbf{w}(t) & \text{otherwise} \end{cases}$$

- **Perceptron Convergence Theorem:** for linearly separable data the algorithm reaches a solution in a finite number of steps, if $\sum \rho_k \rightarrow \infty$ and $\sum \rho_k^2 < \infty$ (for example $\rho_t = c/t$). The solution is not unique.
- It does not converge if the data is not linearly separable.
- **Pocket algorithm:** keep the best weight vector seen so far (most training vectors correct). Works for non-separable data.

Least squares methods

- **MSE:** minimize $J(\mathbf{w}) = E[|y - \mathbf{x}^T \mathbf{w}|^2]$. Solution $\hat{\mathbf{w}} = R_x^{-1} E[\mathbf{x}y]$, with $R_x = E[\mathbf{x}\mathbf{x}^T]$ the **correlation matrix** and $E[\mathbf{x}y]$ the **cross-correlation**. The error is orthogonal to the inputs.
- **LMS / Widrow-Hoff:** same cost, solved iteratively without the distributions: $\hat{\mathbf{w}}(k) = \hat{\mathbf{w}}(k-1) + \rho_k \mathbf{x}_k (y_k - \mathbf{x}_k^T \hat{\mathbf{w}}(k-1))$. With constant ρ it needs $0 < \rho < 2/\text{tr}[R_x]$. Trains the **Adaline** (linear neuron).
- **Sum of error squares:** $J(\mathbf{w}) = \sum_i (y_i - \mathbf{x}_i^T \mathbf{w})^2$. Solution $\hat{\mathbf{w}} = (X^T X)^{-1} X^T \mathbf{y} = X^+ \mathbf{y}$, with X^+ the **pseudoinverse**.
- **Stochastic approximation:** replace an expectation with samples; converges under the same two conditions on ρ_k .

Bias-variance and logistic discrimination

- The best MSE estimate is the conditional mean $E[y | \mathbf{x}]$.
- Error over training sets $D = \text{bias}^2 + \text{variance}$. **Bias:** how far the average model is from $E[y | \mathbf{x}]$. **Variance:** how much the model changes with the data set. Many parameters: low bias, high variance.
- **Logistic discrimination:** the log of the ratio of class posteriors is linear in $\mathbf{x}$. Two classes:

$$P(\omega_2 | \mathbf{x}) = \frac{1}{1 + \exp(w_0 + \mathbf{w}^T \mathbf{x})}$$

and $P(\omega_1 | \mathbf{x}) = 1 - P(\omega_2 | \mathbf{x})$. Parameters come from maximum likelihood. It needs fewer assumptions than LDA (which assumes Gaussian classes).

Traps

- Perceptron cost sums over misclassified vectors only; MSE and least squares use all vectors.

- Step bounds: gradient descent $2/\lambda_{\max}$; LMS $2/\text{tr}[R_x]$.
- LMS trains the Adaline; the perceptron algorithm trains the perceptron.

Lecture 6B: Classifier Evaluation

Confusion matrix (rows = truth, columns = prediction). Slide example: $TN = 116$, $FP = 5$, $FN = 12$, $TP = 23$.

Metric	Formula	Example
Accuracy	$\frac{TP + TN}{\text{all}}$	0.891
Precision = PPV	$\frac{TP}{TP + FP}$	0.821
Recall = Sensitivity = TPR	$\frac{TP}{TP + FN}$	0.657
Specificity = TNR	$\frac{TN}{TN + FP}$	0.959
NPV	$\frac{TN + FN}{FN}$	0.906
FPR = 1 - Specificity	$\frac{FP}{FP + TN}$	0.041
F_1	$\frac{2PR}{P + R}$	0.730
F_β	$\frac{(1 + \beta^2)PR}{\beta^2 P + R}$	$\beta=2$: 0.685
Balanced accuracy	$\frac{\text{Sens.} + \text{Spec.}}{2}$	0.808

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} = 0.669$$

- **Accuracy** is misleading on unbalanced data and when error types have different costs.
- **Precision**: of those predicted positive, how many are positive. **Recall**: of the actual positives, how many were found. Raising one lowers the other.
- F_1 : harmonic mean of precision and recall; ignores TN. F_β : $\beta = 2$ if recall matters twice as much.
- **Sensitivity** low = missed disease cases. **Specificity** low = many false alarms.
- **PPV / NPV** depend on class proportions, so do not use them to compare classifiers across data sets.
- **ROC curve**: TPR against FPR as the threshold varies. **AUC**: area under it; 0.5 is random, 1 is best.
- **MCC**: correlation between predicted and actual, from -1 to 1; uses all four cells.
- Regression: $\text{MSE} = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$ (penalizes large errors more), $\text{MAE} = \frac{1}{N} \sum |\hat{y}_i - y_i|$, Pearson r , $Q^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{m})^2}$.
- Lowering the threshold: sensitivity and FPR go up, specificity goes down.

Slide errors to know: multi-class Recall_P is printed as 0.054 but $10/67 = 0.149$ (balanced accuracy 0.338, not 0.314). The MCC example's stated counts do not match its results. The ROC threshold bullets contradict the figure. Show your computation if these appear.

Traps

- Specificity is not 1 - sensitivity.
- The slide's matrix has TN in the top-left; read the labels.

Lecture 7A: Neural Networks

An **ANN** is a massively parallel distributed processor that stores experiential knowledge. Like the brain: knowledge is acquired by learning, and it is stored in connection strengths (**synaptic weights**).

Biology

- **Neuron**: cell body, dendrites, axon. **Dendrites** carry impulses toward the cell body; the **axon** carries them away.
- **Synapse**: junction between two neurons; neurotransmitters excite or inhibit the receiving (**postsynaptic**) cell. The neuron fires when the summed input exceeds a threshold.
- **Plasticity**: adapting by creating and modifying synapses.
- Brain vs. computer: many slow simple processors vs. few fast complex ones; distributed, content-addressable memory vs. localized; parallel and self-learning vs. sequential stored programs; robust vs. vulnerable. Neural events take milliseconds, silicon nanoseconds. Energy per operation: brain 10^{-10} microwatt, computer 1 microwatt.

Definitions

- **Learning**: the free parameters (weights) are modified through stimulation by the environment. **Learning algorithm**: the procedure that modifies them.
- **Generalization**: correct outputs for inputs not seen in training.
- **Six properties**: input-output mapping, nonlinearity, adaptivity, evidential response, fault tolerance, VLSI implementability.
- **History**: 1943 McCulloch and Pitts; 1949 Hebb (first learning rule); 1958 Rosenblatt (perceptron); 1969 Minsky and Papert (limitations); 1970s dormancy; 1980s return (backpropagation); 2012 deep learning.

Neuron model

Three parts: **synapses** (weights; positive = excitatory, negative = inhibitory), **summer**, **activation (squashing) function**.

$$v_k = \sum_{j=0}^p w_{kj} x_j, \quad y_k = \varphi(v_k)$$

The bias b_k is the weight w_{k0} on a fixed input $x_0 = +1$ (threshold form: $x_0 = -1$, $w_{k0} = \theta_k$). **Logistic function**: $\varphi(v) = \frac{1}{1 + e^{-av}}$; slope at the origin is $a/4$.

Gates and XOR

Step activation: output 1 if $v \geq 0$, else 0. **OR**: $v = x_1 + x_2 - \frac{1}{2}$. **AND**: $v = x_1 + x_2 - \frac{3}{2}$. **XOR** is not linearly separable, so one perceptron cannot compute it (Minsky and Papert). Two layers can:

$x_1 x_2$	y_1 (OR)	y_2 (AND)	$v = y_1 - 2y_2 - \frac{1}{2}$	Out
00	0	0	$-\frac{1}{2}$	0
01, 10	1	0	$\frac{1}{2}$	1
11	1	1	$-\frac{3}{2}$	0

The hidden layer maps the inputs to a space where the classes are linearly separable.

Architectures and learning

- Count only layers of computing neurons; the input layer is not counted.
- **Single-layer feedforward**; **multilayer feedforward** (has **hidden layers**); **fully / partially connected** (**receptive field**); **recurrent** (at least one feedback loop); **lattice**.
- **Supervised**: labeled samples; the **error signal** = desired – actual response adjusts the weights. **Unsupervised (self-organized)**: no teacher. **Reinforcement**: only a score for performance.
- **Learning rules**: error-correction, Hebbian, competitive, Boltzmann.
- **Two-layer perceptron capability**: p hidden step neurons map the input space onto vertices of a unit hypercube; the output neuron cuts it with one hyperplane. It can separate classes made of unions of polyhedral regions, but not every such union.

Traps

- One hidden layer = a two-layer network.
- Logistic slope at the origin is $a/4$, not a .

Lecture 7B: Backpropagation

Training algorithm for the **Multilayer Perceptron (MLP)**; a generalization of LMS. Function signals go forward, error signals go backward. The activation function must be differentiable and nonlinear.

- **Induced local field**: $v_j = \sum_i w_{ji} y_i$. **Output**: $y_j = \varphi(v_j)$. **Error**: $e_j = d_j - y_j$.
- **Instantaneous error**: $\mathcal{E}(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n)$. **Average error**: $\mathcal{E}_{av} = \frac{1}{N} \sum_n \mathcal{E}(n)$ (the cost).
- **Local gradient**: $\delta_j = -\partial \mathcal{E} / \partial v_j$. **Learning rate** η . **Momentum constant** α .
- **Epoch**: one pass over all training examples. **Pattern mode**: update after each example (less storage, faster). **Batch mode**: update after each epoch (more accurate gradient).
- **Fan-in** F_i : number of inputs of neuron i .
- History: Linnainmaa 1970, Dreyfus 1973, Werbos 1974, Parker 1985, LeCun 1985, Rumelhart et al. 1986.

Formulas

$$\Delta w_{ji} = \eta \delta_j y_i, \quad \delta_j = \begin{cases} e_j \varphi'(v_j) & \text{output} \\ \varphi'(v_j) \sum_k \delta_k w_{kj} & \text{hidden} \end{cases}$$

Logistic: $\varphi' = a \varphi(1 - \varphi)$. Tanh, $\varphi = a \tanh(bv)$: $\varphi' = \frac{b}{a} (a - \varphi)(a + \varphi)$. With momentum: $\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n)$.

Algorithm (pattern mode)

1. Initialize weights to small random values.
2. Forward pass: compute v_j , y_j layer by layer.
3. Error: $e_j = d_j - o_j$.
4. Backward pass: output δ first, then hidden δ using the

old w_{kj} .

5. Update all weights: $w_{ji} \leftarrow w_{ji} + \eta \delta_j y_i$ (bias: $y_i = +1$).
6. Repeat over examples (random order each epoch) until the stopping criterion is met.

Worked example

2-2-1 network, logistic ($a = 1$), bias input $+1$, $\eta = 0.5$, $\mathbf{x} = (1, 0)$, $d = 1$. h_1 : bias 0.1, weights 0.4, -0.2 . h_2 : bias -0.1 , weights 0.3, 0.5. Output: bias 0.2, weights 0.6 (from h_1), -0.4 (from h_2).

$$\begin{aligned} v_{h_1} &= 0.1 + 0.4 = 0.5 & y_{h_1} &= 0.6225 \\ v_{h_2} &= -0.1 + 0.3 = 0.2 & y_{h_2} &= 0.5498 \end{aligned}$$

$$\begin{aligned} v_o &= 0.2 + 0.6(0.6225) - 0.4(0.5498) = 0.3535, \text{ so } o = 0.5875. \\ e &= 1 - 0.5875 = 0.4125, \mathcal{E} = \frac{1}{2} e^2 = 0.0851. \end{aligned}$$

$$\delta_o = e o(1 - o) = 0.1000$$

$$\delta_{h_1} = y_{h_1}(1 - y_{h_1}) \delta_o(0.6) = 0.0141$$

$$\delta_{h_2} = y_{h_2}(1 - y_{h_2}) \delta_o(-0.4) = -0.0099$$

New output weights: bias $0.2 + 0.5(0.1)(1) = 0.25$; from h_1 $0.6 + 0.5(0.1)(0.6225) = 0.6311$; from h_2 $-0.4 + 0.0275 = -0.3725$. New h_1 : bias 0.1070, weights 0.4070, -0.2 . New h_2 : bias -0.1049 , weights 0.2951, 0.5. A zero input gives a zero update. Check: new $o = 0.6085$, $\mathcal{E} = 0.0766$.

$$\varphi(0) = 0.5, \varphi(0.2) = 0.5498, \varphi(0.5) = 0.6225, \varphi(1) = 0.7311, \varphi(2) = 0.8808.$$

Practice points

- **Stopping criteria**: gradient norm small; change in $\mathcal{E}_{av}$ per epoch small (0.1 to 1%); $\mathcal{E}_{av}$ small; or generalization performance has peaked.
- **Heuristics**: pattern mode; tanh with $a = 1.716$, $b = 2/3$; targets offset by ϵ from the limits; normalized inputs; initial weights uniform in $\pm 2.4/F_i$; randomized example order.
- Small η : smooth but slow. Large η : may oscillate. Momentum speeds learning while keeping stability.
- **Decision rule**: assign the class with the largest output.
- **Convergence**: gradient descent on the error surface; reaches a local minimum, not necessarily the global one.
- **Overfitting**: the network memorizes the training data and generalizes poorly.
- **Training set size (Baum and Haussler)**: in practice $N \geq W/\epsilon$ (W weights, ϵ allowed error); about 10 times the weights for 10% error.

Traps

- A hidden neuron has no e_j (no desired response).
- Batch mode has the more accurate gradient; pattern mode is faster.
- With a linear activation an MLP reduces to a single layer.
- Slides: derivation uses bias input $+1$; the summary slide uses $y_0 = -1$. Use what the problem states.

Full backpropagation derivation

Notation: neuron i feeds neuron j , which feeds neuron k . $w_{ji}(n)$ is the weight from i to j at iteration n . Neuron j has p inputs plus a fixed input $y_0 = +1$ with weight $w_{j0} = b_j$. φ is differentiable and nonlinear.

Step 1. Definitions.

$$v_j(n) = \sum_{i=0}^p w_{ji}(n) y_i(n), \quad y_j(n) = \varphi(v_j(n)), \quad e_j(n) = d_j(n) - y_j(n) \quad (\text{D1})$$

$$\mathcal{E}(n) = \frac{1}{2} \sum_{j \in C} e_j^2(n), \quad \mathcal{E}_{av} = \frac{1}{N} \sum_{n=1}^N \mathcal{E}(n), \quad \Delta w_{ji}(n) = -\eta \frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} \quad (\text{gradient descent}) \quad (\text{D2})$$

Step 2. Output neuron. w_{ji} affects $\mathcal{E}$ only through $v_j \rightarrow y_j \rightarrow e_j$, so by the chain rule

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \underbrace{\frac{\partial \mathcal{E}(n)}{\partial e_j(n)}}_{e_j(n)} \cdot \underbrace{\frac{\partial e_j(n)}{\partial y_j(n)}}_{-1} \cdot \underbrace{\frac{\partial y_j(n)}{\partial v_j(n)}}_{\varphi'(v_j(n))} \cdot \underbrace{\frac{\partial v_j(n)}{\partial w_{ji}(n)}}_{y_i(n)} = -e_j(n) \varphi'(v_j(n)) y_i(n) \quad (\text{D3})$$

$$\Delta w_{ji}(n) = \eta e_j(n) \varphi'(v_j(n)) y_i(n) = \eta \delta_j(n) y_i(n), \quad \text{where} \quad \delta_j(n) = -\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = e_j(n) \varphi'(v_j(n)) \quad (\text{D4})$$

Step 3. Hidden neuron. A hidden neuron has no desired response, so e_j does not exist. $\Delta w_{ji} = \eta \delta_j y_i$ still holds; only δ_j needs a new expression:

$$\delta_j(n) = -\frac{\partial \mathcal{E}(n)}{\partial v_j(n)} = -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} = -\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \varphi'(v_j(n)) \quad (\text{D5})$$

y_j affects $\mathcal{E} = \frac{1}{2} \sum_k e_k^2$ only through the v_k of the neurons it feeds. With $e_k = d_k - \varphi(v_k)$ and $v_k = \sum_j w_{kj} y_j$:

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = \sum_k e_k(n) \underbrace{\frac{\partial e_k(n)}{\partial v_k(n)}}_{-\varphi'(v_k(n))} \cdot \underbrace{\frac{\partial v_k(n)}{\partial y_j(n)}}_{w_{kj}(n)} = -\sum_k e_k(n) \underbrace{\varphi'(v_k(n)) w_{kj}(n)}_{\delta_k(n)} = -\sum_k \delta_k(n) w_{kj}(n) \quad (\text{D6})$$

The two minus signs cancel:

$$\delta_j(n) = \varphi'(v_j(n)) \sum_k \delta_k(n) w_{kj}(n) \quad (\text{D7})$$

The same recursion holds when the neurons k are hidden; their δ_k come from the layer to their right. This is why the computation runs backward from the output.

Step 4. Activation derivative.

$$\text{Logistic: } \varphi(v) = \frac{1}{1 + e^{-av}} \Rightarrow \varphi'(v) = \frac{a e^{-av}}{(1 + e^{-av})^2} = a \cdot \frac{1}{1 + e^{-av}} \cdot \frac{e^{-av}}{1 + e^{-av}} = a \varphi(v) [1 - \varphi(v)] \quad (\text{D8})$$

$$\text{Tanh: } \varphi(v) = a \tanh(bv) \Rightarrow \varphi'(v) = ab \operatorname{sech}^2(bv) = ab [1 - \tanh^2(bv)] = \frac{b}{a} [a - \varphi(v)] [a + \varphi(v)] \quad (\text{D9})$$

	Output neuron (o_j = output)	Hidden neuron
Logistic	$\delta_j = a (d_j - o_j) o_j (1 - o_j)$	$\delta_j = a y_j (1 - y_j) \sum_k \delta_k w_{kj}$
Tanh	$\delta_j = \frac{b}{a} (d_j - o_j) (a^2 - o_j^2)$	$\delta_j = \frac{b}{a} (a^2 - y_j^2) \sum_k \delta_k w_{kj}$

Step 5. Generalized delta rule (momentum $\alpha > 0$).

$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n), \quad w_{ji}(n+1) = w_{ji}(n) + \Delta w_{ji}(n), \quad \Delta b_j = \eta \delta_j \quad (y_0 = +1) \quad (\text{D10})$$

Where marks are lost. (1) Signs: $\partial e_j / \partial y_j = -1$ and $\partial e_k / \partial v_k = -\varphi'(v_k)$. (2) Dropping the sum over k . (3) Index order: w_{kj} goes from j to k ; hidden δ_j uses the weights leaving j . (4) Using e_j for a hidden neuron. (5) δ_j is defined with respect to v_j , not y_j or w_{ji} . (6) Not stating that φ must be differentiable and nonlinear.

Proofs

$g(n)$ cost from start to n ; $h(n)$ heuristic; $h^*(n)$ true cost from n to a goal; $f = g + h$; $c(n, n')$ step cost; C^* optimal cost.

Proof 1. A^* with an admissible h is optimal.

Suppose A^* selects a suboptimal goal G_2 , so $g(G_2) > C^*$. Some node n on an optimal path is still on the frontier. Admissibility gives $f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*$. A^* chose G_2 over n , so $f(G_2) \leq f(n) \leq C^*$. Since $h(G_2) = 0$, $g(G_2) = f(G_2) \leq C^*$. Contradiction. ■

Proof 2. Consistency implies admissibility.

Induction on k , the number of steps on a cheapest path from n to a goal. Base $k = 0$: n is a goal, $h(n) = 0 = h^*(n)$. Step: let n' be the first successor on a cheapest path of $k + 1$ steps. Then

$$h(n) \leq c(n, n') + h(n') \leq c(n, n') + h^*(n') = h^*(n)$$

(consistency, then the induction hypothesis). ■

Proof 3. Consistent h makes f non-decreasing.

$$f(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n). \quad \blacksquare$$

Proof 4. Admissible does not imply consistent.

$S \rightarrow A$ costs 1, $A \rightarrow G$ costs 3; $h(S) = 4$, $h(A) = 1$, $h(G) = 0$. Admissible ($h^*(S) = 4$, $h^*(A) = 3$), but $h(S) = 4 > c(S, A) + h(A) = 2$.

Proof 5. 8-puzzle heuristics.

h_1 : every misplaced tile must move at least once and a move moves one tile, so $h_1 \leq h^*$. h_2 : a move shifts one tile one square, reducing the distance sum by at most 1, so $h_2 \leq h^*$. Consistent: $h_2(n) - h_2(n') \leq 1 = c(n, n')$. Dominance: a misplaced tile adds exactly 1 to h_1 and at least 1 to h_2 .

Proof 6. Max of admissible heuristics.

If each $h_i(n) \leq h^*(n)$ then $\max_i h_i(n) \leq h^*(n)$, and $\max_i h_i(n) \geq h_i(n)$ for every i . ■

Proof 7. Special cases.

A^* with $h = 0$ has $f = g$: this is UCS, and $h = 0$ is admissible, so UCS is optimal. BFS is UCS with $g(n) = \text{DEPTH}(n)$. Greedy is not optimal: Arad to Bucharest gives 450 instead of 418.

Proof 8. UCS is optimal when step costs are $\geq \epsilon > 0$.

g never decreases along a path. UCS selects the frontier node with the lowest g , so when goal G is selected every other frontier node m has $g(m) \geq g(G)$. Any other solution passes through some such m and costs at least $g(m)$. The bound $\epsilon > 0$ makes g grow without limit on infinite paths, so G is eventually selected. ■

Proof 9. IDS: time $O(b^d)$, space $O(bd)$.

Depth- d nodes are generated once, depth $d - 1$ twice, ..., depth 1 d times: $N = db + (d - 1)b^2 + \dots + b^d = O(b^d)$. Each iteration is depth-first, storing at most bd nodes. ■

Proof 10. Alpha-beta returns the minimax value.

At a MIN node with running value $v \leq \alpha$, its final value is $\leq v \leq \alpha$. The MAX ancestor already has an option worth α , so this branch cannot raise its maximum. The remaining children cannot affect the root. The MAX case ($v \geq \beta$) is symmetric. ■

Proof 11. $\mathbf{w}$ is orthogonal to the hyperplane; distance is $|g(\mathbf{x})|/\|\mathbf{w}\|$.

For $\mathbf{x}_A, \mathbf{x}_B$ on the hyperplane, subtracting $\mathbf{w}^\top \mathbf{x}_A + w_0 = 0$ and $\mathbf{w}^\top \mathbf{x}_B + w_0 = 0$ gives $\mathbf{w}^\top (\mathbf{x}_A - \mathbf{x}_B) = 0$. Write $\mathbf{x} = \mathbf{x}_p + z\mathbf{w}/\|\mathbf{w}\|$ with $\mathbf{x}_p$ on the hyperplane: $g(\mathbf{x}) = 0 + z\|\mathbf{w}\|$, so $z = g(\mathbf{x})/\|\mathbf{w}\|$. ■

Proof 12. XOR is not linearly separable.

Output 1 iff $w_1x_1 + w_2x_2 + w_0 \geq 0$. Required: (a) $w_0 < 0$; (b) $w_1 + w_0 \geq 0$; (c) $w_2 + w_0 \geq 0$; (d) $w_1 + w_2 + w_0 < 0$. Adding (b) and (c): $w_1 + w_2 + w_0 \geq -w_0 > 0$, contradicting (d). ■

Proof 13. Perceptron cost and update.

A misclassified $\mathbf{x} \in \omega_1$ has $\mathbf{w}^\top \mathbf{x} < 0$ and $\delta_x = -1$; a misclassified $\mathbf{x} \in \omega_2$ has $\mathbf{w}^\top \mathbf{x} > 0$ and $\delta_x = +1$. Each term $\delta_x \mathbf{w}^\top \mathbf{x}$ is positive, so $J \geq 0$. $\partial J / \partial \mathbf{w} = \sum_{\mathbf{x} \in Y} \delta_x \mathbf{x}$, so gradient descent gives $\mathbf{w}(t+1) = \mathbf{w}(t) - \rho_t \sum_{\mathbf{x} \in Y} \delta_x \mathbf{x}$. ■

Proof 14. Perceptron Convergence Theorem (outline).

Let $\mathbf{w}^*$ be a solution, $\alpha > 0$, and $\mathbf{s} = \sum_{\mathbf{x} \in Y} \delta_x \mathbf{x}$. From the update,

$$\begin{aligned} \|\mathbf{w}(t+1) - \alpha \mathbf{w}^*\|^2 &= \|\mathbf{w}(t) - \alpha \mathbf{w}^*\|^2 + \rho_t^2 \|\mathbf{s}\|^2 \\ &\quad - 2\rho_t (\mathbf{w}(t) - \alpha \mathbf{w}^*)^\top \mathbf{s}. \end{aligned}$$

$\mathbf{w}(t)^\top \mathbf{s} \geq 0$ for misclassified vectors, so

$$\|\mathbf{w}(t+1) - \alpha \mathbf{w}^*\|^2 \leq \|\mathbf{w}(t) - \alpha \mathbf{w}^*\|^2 + \rho_t^2 \beta^2 - 2\rho_t \alpha |\gamma|,$$

with $\beta^2 = \max \|\mathbf{s}\|^2$ and $\gamma = \max \mathbf{w}^* \mathbf{s} < 0$ over subsets of the training set. Choose $\alpha = \beta^2 / (2|\gamma|)$ and unroll to $t = 0$:

$$\|\mathbf{w}(t+1) - \alpha \mathbf{w}^*\|^2 \leq \|\mathbf{w}(0) - \alpha \mathbf{w}^*\|^2 + \beta^2 \left(\sum_{k=0}^t \rho_k^2 - \sum_{k=0}^t \rho_k \right).$$

If $\sum \rho_k \rightarrow \infty$ and $\sum \rho_k^2 < \infty$, the right side reaches zero at a finite t_0 , so $\mathbf{w}(t_0 + 1) = \alpha \mathbf{w}^*$. ■

Proof 15. MSE and least squares solutions.

$\partial J / \partial \mathbf{w} = -2E[\mathbf{x}(y - \mathbf{x}^\top \mathbf{w})] = \mathbf{0}$ gives $E[\mathbf{x}\mathbf{x}^\top] \mathbf{w} = E[\mathbf{x}y]$, so $\hat{\mathbf{w}} = R_x^{-1} E[\mathbf{x}y]$. For $J = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$: $-2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$ gives $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. ■

Proof 16. Gradient descent converges for $0 < \mu < 2/\lambda_{\max}$.

With $R\theta^0 = \mathbf{p}$, the update gives $\theta(k+1) - \theta^0 = (I - \mu R)(\theta(k) - \theta^0)$. In the eigenbasis of R each component is scaled by $(1 - \mu\lambda_i)$, which decays iff $|1 - \mu\lambda_i| < 1$, that is $0 < \mu < 2/\lambda_i$. The largest eigenvalue is the binding case. ■

Proof 17. Bias-variance decomposition.

Let $\bar{g} = E_D[g]$, $t = E[y | \mathbf{x}]$. $(g - t)^2 = (g - \bar{g})^2 + (\bar{g} - t)^2 + 2(g - \bar{g})(\bar{g} - t)$. Taking E_D , the cross term vanishes because $E_D[g - \bar{g}] = 0$. Result: $(\bar{g} - t)^2 + E_D[(g - \bar{g})^2] = \text{bias}^2 + \text{variance}$. ■

Proof 18. $F_1 = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$.

$$\frac{2PR}{P + R} = \frac{2TP^2}{TP(2TP + FP + FN)} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

$$\text{using } P = \frac{TP}{TP + FP}, R = \frac{TP}{TP + FN}. \quad \blacksquare$$

Proofs 19, 20, 21 (logistic derivative, delta rule for an output neuron, local gradient of a hidden neuron) are Steps 4, 2, and 3 of the derivation.

Proof 22. Linear activations collapse to one layer.

With $\varphi(v) = v$: $\mathbf{o} = W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$, a single weight matrix. So only linear maps are realized and XOR cannot

be solved. ■

Proof 23. Stochastic approximation with $\rho_k = 1/k$ gives the sample mean.

$\hat{\mathbf{w}}(k) = \frac{k-1}{k}\hat{\mathbf{w}}(k-1) + \frac{1}{k}\mathbf{x}_k$. If $\hat{\mathbf{w}}(k-1) = \frac{1}{k-1}\sum_{r=1}^{k-1}\mathbf{x}_r$ then $\hat{\mathbf{w}}(k) = \frac{1}{k}\sum_{r=1}^k\mathbf{x}_r$. Base: $\hat{\mathbf{w}}(1) = \mathbf{x}_1$. ■